Course ID: AIMLCZG523

MLOps Assignment 1

Name: Swarnali Deb

Student ID: 2025ae05588

GitHub Repository Link

https://github.com/2025ae05588/ml-assignment1

Demonstration Video

Heart Disease MLOps - Video Walkthrough (MP4)

01 Project overview and architecture

This project develops a heart disease classification system using the UCI Cleveland dataset. The workflow covers data preparation, model comparison, experiment tracking and API deployment on the Prayogshala VM. Testing and monitoring support the application from development through deployment.

The model predicts the recorded heart disease label from 13 patient-health features. Its purpose is to demonstrate an end-to-end MLOps workflow. Clinical diagnosis and prediction of future disease risk are outside the scope of this project.

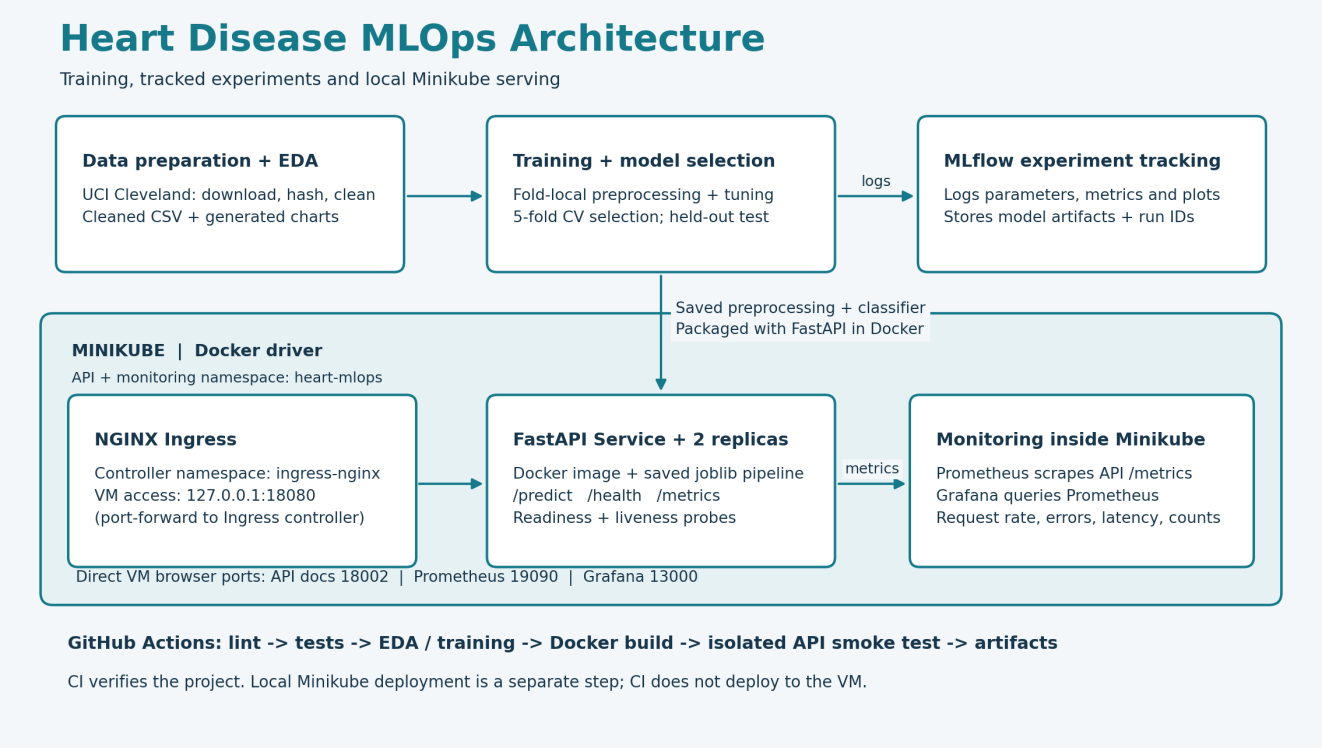


Figure 1. The workflow from data preparation to Minikube deployment. MLflow records experiments. NGINX Ingress, API replicas and monitoring run inside Minikube. GitHub Actions verifies the project; local deployment is separate.

02 Dataset acquisition and preparation

Source: UCI Heart Disease dataset

The Cleveland subset contains 303 records, 13 predictors and a diagnosis column. The diagnosis values were mapped to a binary target: 0 indicates absence of the recorded disease label, while values 1-4 indicate presence. The download script checks the raw file and writes the cleaned dataset. Source details and checksums are recorded in data/metadata.json.

Property	Observed result
Raw / cleaned rows	303 / 303
Exact duplicates removed	0
Missing values	ca: 4; thal: 2; all other predictors: 0
Label mapping	num = 0 -> absent; num = 1, 2, 3, 4 -> present
Class counts	Absent: 164 (54.1%); present: 139 (45.9%)

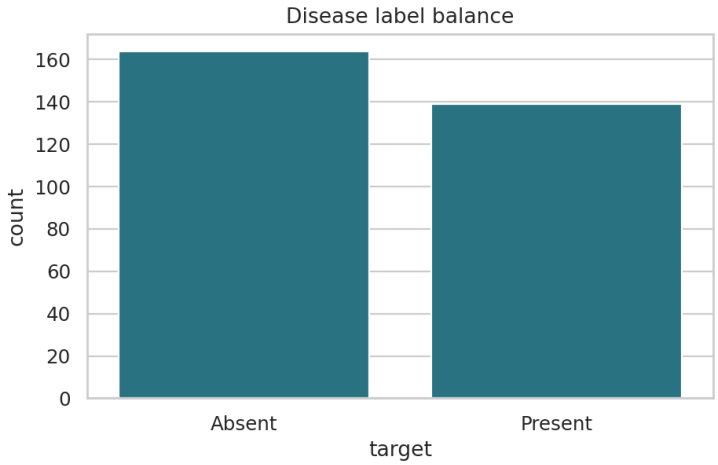


Figure 2. Neither class overwhelmingly dominates; precision, recall and ROC-AUC remain necessary to interpret different classification errors.

Raw-data SHA-256: a74b7efa387bc9d108d7d0115d831fe9b414b29ae7124f331b622b4efa0427c8

Cleaned-data SHA-256: e81973d3356c231ed8903aab16f5faa72d9679eb8ef6dc98a602b25df3f5d23d

Missing values occur in ca and thal. The '?' markers were converted to missing values and retained for imputation during training. Category checks preserve the original UCI coding, including cp 1-4, slope 1-3, ca 0-3 and thal 3/6/7. The hashes above identify the raw and cleaned files used in the experiment.

03 Exploratory data analysis

Exploratory analysis examines class balance, missing values and feature distributions before model development. scripts/eda.py produces the charts and summary tables; notebooks/01_eda.ipynb provides the notebook version. The class counts are 164 absent and 139 present, indicating a modest difference rather than severe class imbalance.

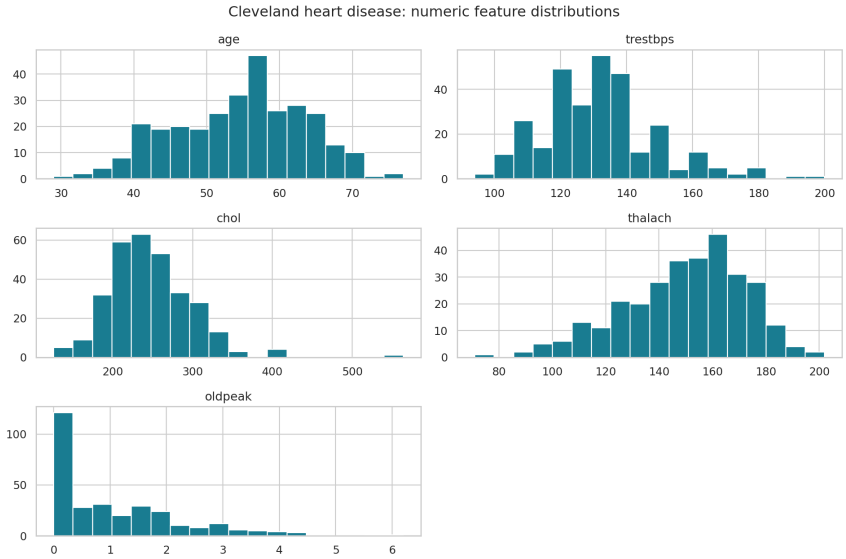


Figure 3. Numerical predictor distributions. Distribution shape and unusual values motivate checking data quality, rather than deleting observations automatically.

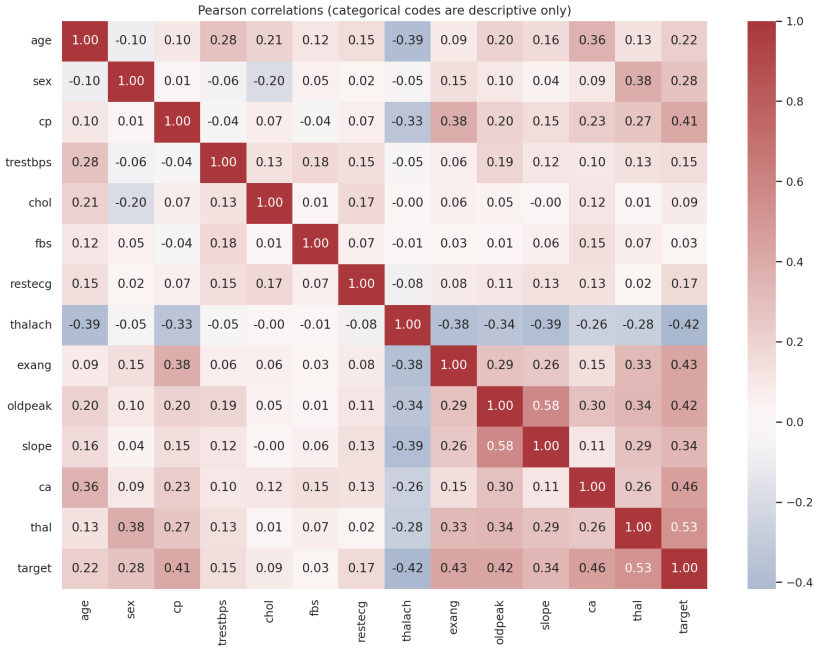


Figure 4. Pearson correlations, including numerically coded categories. Category-code correlations are descriptive; they are not continuous clinical effects or causal claims.

All 13 predictors were retained. Correlations were used to describe the data, not to select features using the test set. Imputation and scaling are fitted during training rather than EDA, keeping validation and test information separate from the learned transformations.

04 Feature pipeline and model selection

The model pipeline combines a ColumnTransformer with a classifier. Five numerical features - age, trestbps, chol, thalach and oldpeak - use median imputation and standard scaling. The remaining eight features use most-frequent imputation and one-hot encoding. This separates continuous measurements from category codes and keeps preprocessing consistent across the two classifiers.

A stratified 80:20 split produces 242 training records and 61 test records, with random seed 42. Model tuning uses five-fold shuffled stratified cross-validation within the training set. Each fold fits its own preprocessing, preventing information from the validation fold from entering the training process.

Model	Search space / configuration
Logistic Regression	C in {0.1, 1, 10}; max_iter=2000; seed=42
Random Forest	150 trees; max_depth in {3, None}; min_samples_leaf in {1, 4}; seed=42
Comparison	7 configurations, 5 folds; common preprocessing and scoring
Selection	Highest mean CV ROC-AUC; held-out set excluded from selection

Best configuration from each estimator

Estimator	Accuracy	Precision	Recall	ROC-AUC
logistic regression	0.8471	0.8768	0.7834	0.9025
random forest	0.8138	0.8348	0.7462	0.8992

Logistic Regression achieved the higher mean CV ROC-AUC and was selected with C=1.0. The best Random Forest used max_depth=3 and min_samples_leaf=1. Although the difference is small, the validation results favour Logistic Regression; the test set was not used to choose between the models.

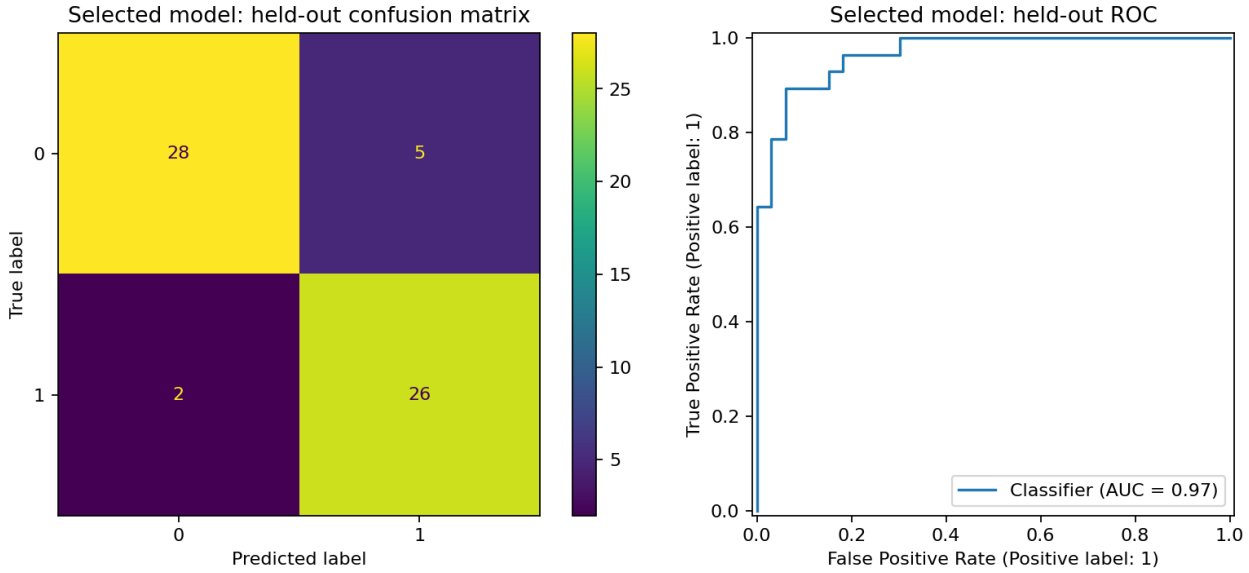
Accuracy, precision, recall and ROC-AUC were recorded for every configuration. Fold means and standard deviations describe both performance and variation. The complete search results are stored in artifacts/logistic_regression_cv.csv and artifacts/random_forest_cv.csv.

artifacts/split.json records the exact training and test indices. The search raises an error if a fit fails, so unsuccessful configurations cannot pass unnoticed.

05 Held-out evaluation and limitations

The selected model was evaluated on the held-out test set after cross-validation. A fixed threshold of 0.5 converts predicted probabilities into classes. Neither model selection nor threshold adjustment used the test outcomes.

Held-out metric	Value	Interpretation
Accuracy	0.8852	Fraction of all predictions correct
Precision	0.8387	Correct positives among positive predictions
Recall	0.9286	Detected positives among actual positives
ROC-AUC	0.9665	Discrimination across decision thresholds



Figures 5-6. Held-out confusion matrix and ROC curve for the selected pipeline.

The confusion matrix contains 28 true negatives, 26 true positives, 5 false positives and 2 false negatives. Recall of 0.9286 indicates that most positive cases were detected on this split. However, the test set has only 61 records, so a small change in predictions would noticeably affect the metrics.

The results describe performance on this dataset and split. They do not establish performance in another hospital or population. The data is historical, and the probabilities have not been calibrated or externally validated for clinical use.

Further evaluation should include an independent dataset, subgroup comparisons and calibration analysis. Threshold choices should reflect the intended use and error costs. The saved model is the evaluated training-set model; it was not refitted using the test records.

06 Experiment tracking and model packaging

MLflow records the experiment settings, results and model artifacts. Run details are stored in a local SQLite database, mlflow.db. Seven candidate runs contain hyperparameters, the seed, data hash, CV metrics and EDA plots. The selected-model-heldout-evaluation run contains the test results, evaluation plots and fitted pipeline.

MLflow evaluation run ID: cab705edb7524048a8ad5b31be8316c4

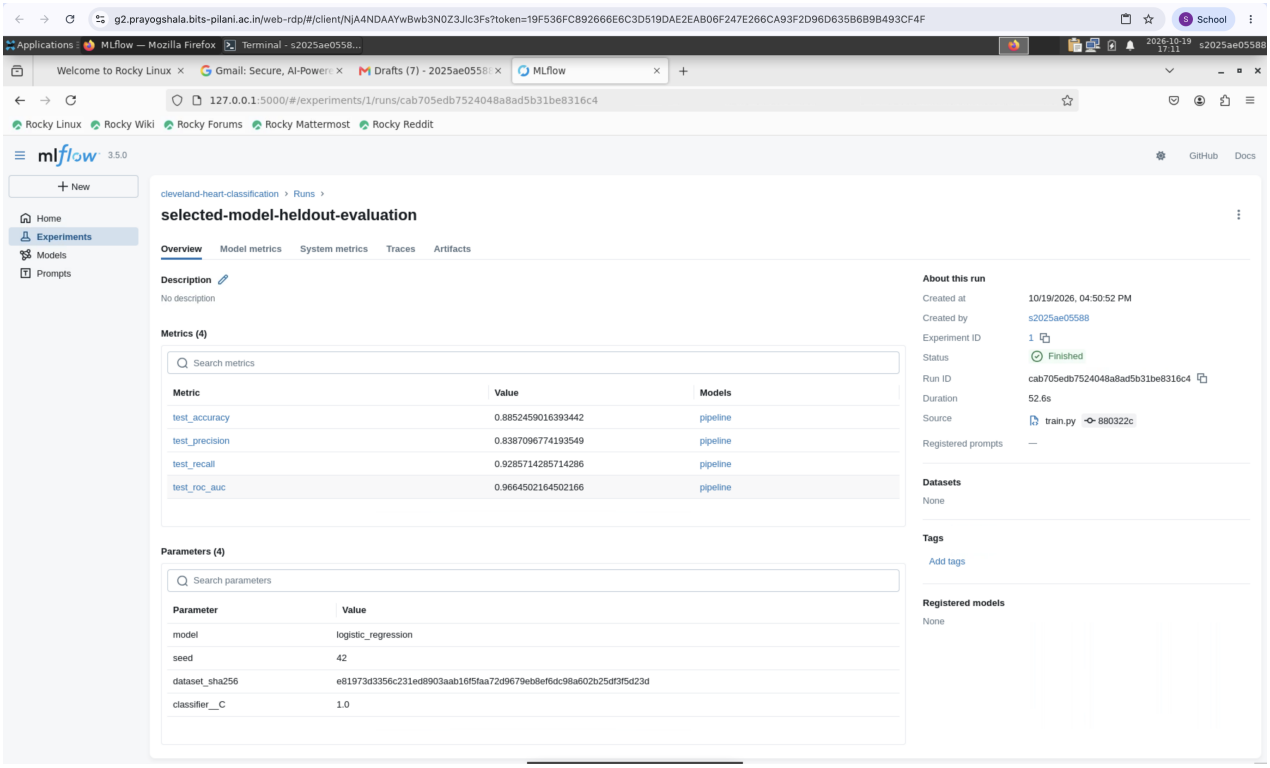


Figure 7. MLflow evaluation results from the Prayogshala VM. The run ID and metrics match artifacts/metrics.json and the API screenshots.

The fitted preprocessing and classifier are packaged together in artifacts/model.joblib. The accompanying metrics.json records the model version, evaluation metrics and dataset hash. FastAPI loads the pipeline at startup and applies it directly to prediction requests, avoiding a separate implementation of preprocessing.

requirements.txt specifies the direct dependency versions, while requirements-lock.txt provides a resolved-environment reference. Python 3.12 or 3.13 is recommended for local setup; Docker and CI use Python 3.13. Retaining the split indices, seed and package versions helps explain and reproduce the reported results.

The model, metrics, data hashes and split file form the core experiment record. MLflow tracking files are also available through CI artifacts. Since joblib deserialization can execute Python objects, model files should only be loaded from trusted sources.

```
mlflow ui --backend-store-uri sqlite:///mlflow.db \
  --host 127.0.0.1 --port 5000
```

07 Inference API and Docker isolation

FastAPI exposes /predict, /health and /metrics, with interactive documentation at /docs. Pydantic validates required fields, category codes, numeric ranges and unexpected fields. Explicit nulls are allowed for ca and thal so that the fitted imputer can handle missing values.

A prediction response contains the class, disease probability, confidence and model version. Confidence is the probability assigned to the predicted class. It is not a guarantee of correctness or a calibrated clinical score; the response includes an educational-use note.

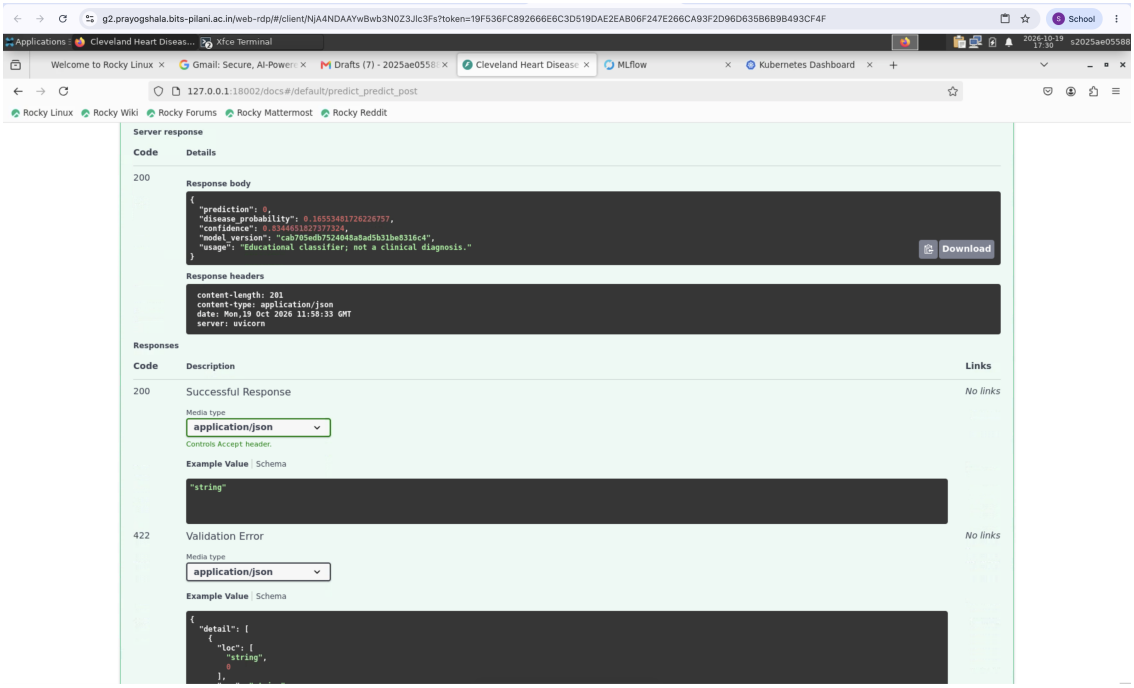


Figure 8. HTTP 200 prediction response. The model version matches the MLflow evaluation run. The example returns class 0 with confidence about 0.8345.

The Docker image is based on python:3.13-slim and includes the application, saved model and metadata. The service runs on port 8000 as a non-root user. A healthcheck calls /health, and a missing model causes startup to fail rather than leaving an unusable service running.

scripts/docker_verify.sh checks serving in an isolated container. It builds the image, starts a temporary container on port 8001 and calls the health and prediction endpoints. Logs are saved before the container is removed. The build output is included in screenshots/docker-build-vm.png.

Request logs contain the HTTP method, route, response status and duration. Patient payloads are excluded because they are unnecessary for these operational checks and could expose sensitive information.

```
curl -fsS http://127.0.0.1:18080/predict \
  -H 'Content-Type: application/json' \
  --data @examples/patient.json
```

08 Automated tests and continuous integration

Unit tests check data cleaning, preprocessing, model serialization and API behavior. A deployment regression test checks image tagging and Ingress readiness using mocked commands. These tests complement model evaluation by checking whether the surrounding software behaves as expected.

Area	Checks implemented
Data	Target mapping, missing-value preservation, duplicate handling, invalid targets/categories
Model	Training-only imputation statistics; serialized pipeline prediction equivalence
API	Valid prediction, health, metrics, invalid/missing/extra fields and nullable features
Deployment	Mocked command order, content-tagged image, admission readiness and retry

GitHub Actions runs dependency installation, lint, data preparation, tests, EDA, training and Docker verification after a push. A failure in lint, tests or training stops subsequent normal steps. Artifact upload still runs to retain available outputs for investigation. Kubernetes deployment is performed separately; the workflow contains no VM cluster credentials.

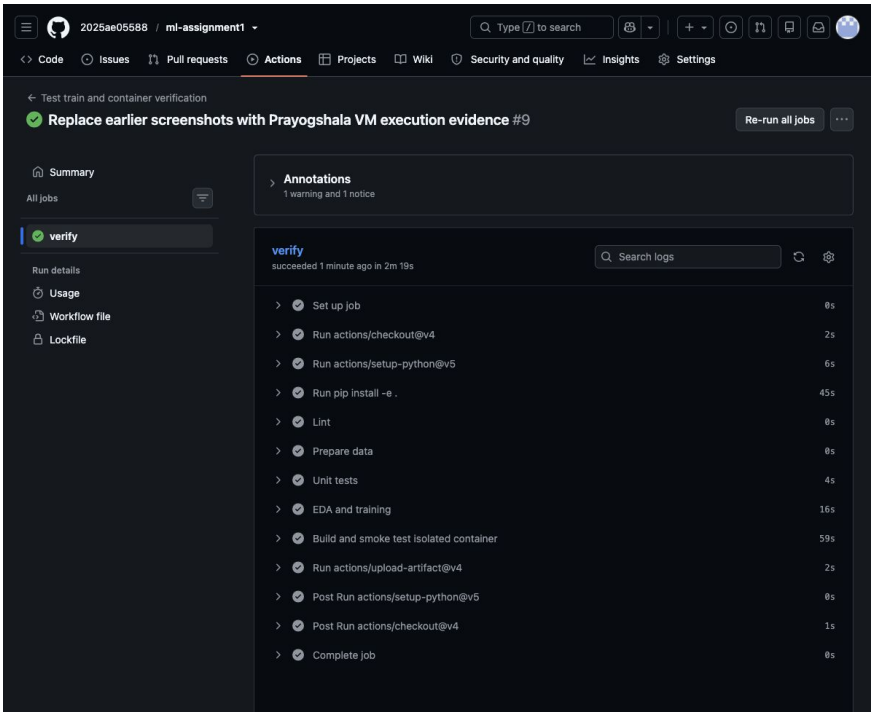


Figure 9. Successful GitHub Actions workflow showing completed validation, training, Docker verification and artifact upload steps.

Open a successful GitHub Actions workflow

Each workflow retains model outputs, metrics, test results and tracking files as downloadable artifacts. These files provide a record of the execution and support troubleshooting when a check fails.

09 Minikube deployment and monitoring

The application runs on Minikube with the Docker driver under profile heart-prayogshala. scripts/deploy_local.sh loads a content-tagged image and waits for the NGINX controller and admission jobs before applying Ingress. Temporary admission failures are retried. The API and monitoring use namespace heart-mlops; the controller uses ingress-nginx.

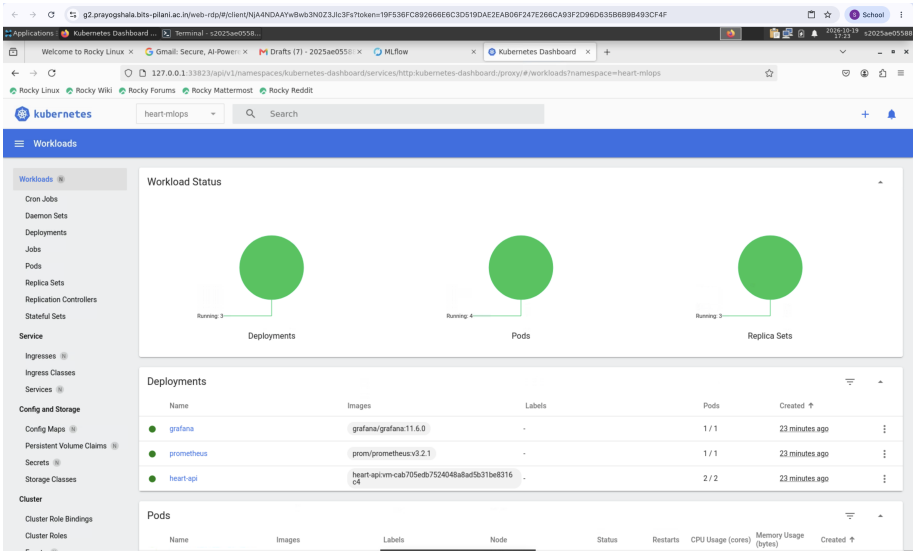


Figure 10. Minikube Dashboard: two API replicas and ready monitoring workloads. Readiness/liveness probes, resource limits, non-root API execution and dropped capabilities are configured in deployment/kubernetes.yaml.

Prometheus scrapes the API Service every five seconds, and Grafana displays the resulting metrics. Both services run inside Minikube. Requests through Ingress reach the API replicas, whereas a direct Service port-forward can select one pod. Per-pod scraping would be preferable for production monitoring because a load-balanced Service does not preserve a separate counter identity for each replica.

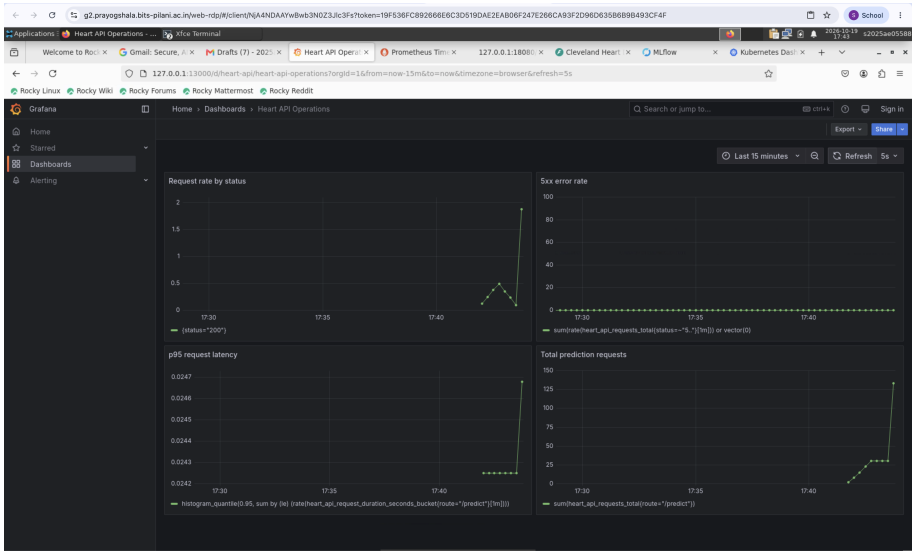


Figure 11. VM request rate, 5xx errors, p95 latency and prediction counts after traffic generation. The Prometheus target-UP capture is also in screenshots/.

The dashboard reports service activity rather than model quality. Data-drift detection, clinical-performance monitoring, TLS termination, authentication, production alerts and automated retraining are not implemented. Anonymous Grafana access and local port-forwards are demonstration settings that would need to be reviewed before public deployment.

10 Reproduction, access and submission links

The setup requires Git, Python 3.12 or 3.13, Docker with Compose, and Minikube. Docker must be running, with at least two CPUs and 3 GB available for the cluster in addition to host resources. Internet access is needed to download dependencies, data and container images. The commands below reproduce the main workflow.

```
git clone https://github.com/2025ae05588/ml-assignment1.git
cd ml-assignment1
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -e .
python scripts/download_data.py
python scripts/eda.py
ruff check .
pytest -q
python scripts/train.py
bash scripts/docker_verify.sh
bash scripts/deploy_local.sh
```

Run each port-forward in a separate terminal from the repository root. In every terminal, first set export KUBECONFIG="$PWD/deployment/kubeconfig". Keep the terminals running while accessing the browser pages.

```
minikube -p heart-prayogshala kubectl -- -n heart-mlops \
  port-forward service/heart-api 18002:80
minikube -p heart-prayogshala kubectl -- -n heart-mlops \
  port-forward service/prometheus 19090:9090
minikube -p heart-prayogshala kubectl -- -n heart-mlops \
  port-forward service/grafana 13000:3000
minikube -p heart-prayogshala kubectl -- -n ingress-nginx \
  port-forward service/ingress-nginx-controller 18080:80
```

VM-local URL	Use
http://127.0.0.1:18002/docs	Interactive JSON prediction
http://127.0.0.1:18080/health	Ingress verification
http://127.0.0.1:19090/targets	Prometheus target health
http://127.0.0.1:13000/d/heart-api/heart-api-operations	Grafana dashboard

Local URLs refer to the machine running the forwards; no public API URL is claimed. Keep forwards running while viewing. Stop with Ctrl+C; stop the cluster with minikube -p heart-prayogshala stop. MLflow startup is on page 6.

Repository and full setup instructions

Video Walkthrough MP4

If GitHub does not play the video inline, use Download raw file on its file page. The repository also contains code, cleaned data, notebook/scripts, model and metrics, tests, workflow, Dockerfile, manifests, screenshots and this report.